

Cascaded Semantic Joins over Structured and Unstructured Data

Anna Remizova, Sihem Amer-Yahia, Silviu Maniu
CNRS, Univ. Grenoble Alpes
Grenoble, France

ABSTRACT

Questions that combine conditions on structured attributes (a movie’s year or genre) with conditions on unstructured text (its reviews) require a large language model (LLM) as a semantic operator. SUQL evaluates the semantic condition with one LLM call per candidate row, which is accurate but expensive; the semantic join of Trummer packs many rows into each call, which is cheaper but less accurate. We propose two cascaded architectures, CASUQL and CASTRUMMER, that run the structured filter in the DBMS, score the semantic condition with a cheap LLM, and escalate to an expensive LLM only the rows inside an ambiguity band. The band is calibrated with Beta-posterior credible bounds on the recall and precision of the cheap stage, estimated from a small score-stratified sample. On *IMDb* and *Amazon Fashion* (ten questions, ten repetitions each), the calibrated cascades lose no recall: CASUQL matches the F1 of SUQL (−0.2%, +2.1%) while sending fewer rows to the expensive model. CASTRUMMER improves the F1 of TRUMMER by 110% and 45%, issues 81–83% fewer LLM calls than SUQL, and lies on the cost–F1 Pareto front on both datasets. An ablation attributes its quality gain to structural filtering and batched expensive verification, and a cost model states when the cheap stage itself saves time.

KEYWORDS

LLM query processing, structured-unstructured query language, cascaded inference, semantic operators, cost-based optimization

1 INTRODUCTION

Consider the question “Which movies released in 1998 have reviews expressing an overall negative opinion?” It needs (i) a structured filter over a year column and (ii) a semantic filter over free-text reviews. No structured query language can express the second half: a negative opinion has no fixed vocabulary (negation, sarcasm and implication all express it) and no schema to index it by. It therefore has to be delegated to a large language model (LLM) used as a query operator.

Two strategies exist. SUQL [7] compiles the structured half into SQL and calls the LLM once per surviving row, which gives high quality but a number of calls that grows with the candidates. Trummer [12] has the LLM join blocks of the two collections in one call, which needs far fewer calls but is less accurate, because one call must match rows, check the structured condition and judge the text. We build on both and add a cascade: a cheap stage scores each unit of work, and only units inside a calibrated ambiguity band reach the expensive stage. Our contributions, and what they buy, are:

- **Calibrated cascades, CASUQL and CASTRUMMER.** Both run σ_5 in the DBMS first. The ambiguity band is derived from Beta-posterior credible bounds on the cheap stage’s

recall and precision, computed from 20 labeled rows stratified by cheap score, whose expensive answers are reused. A weak calibration can only cost money, never quality (§4.2).

- **Quality with fewer calls.** CASUQL reproduces the quality of SUQL while sending 5–14% fewer rows to the expensive model, with no loss of recall. CASTRUMMER improves the F1 of TRUMMER by 110% on *IMDb* and 45% on *Amazon Fashion* with 28% and 8% fewer calls, issues 81–83% fewer calls than SUQL, and sits on the cost–F1 Pareto front of both datasets (§5.2).
- **An explicit account of when the cascade pays off.** A cost model gives a break-even condition for the cheap stage, and an ablation separates the contribution of the cheap stage from that of structural filtering and batched verification (§4.1, §5.2).

The rest of the paper reviews related work (§2), the query model and baselines (§3), the cascaded architectures and their calibration (§4), and the experiments (§5).

2 RELATED WORK

Avoiding LLM calls on rows a cheap check could reject. SUQL [7] extends SQL with a free-text answer() function and shows that evaluating structured predicates before semantic ones is the dominant lever for controlling LLM cost, since it shrinks the candidate pool before the expensive operator. Our SUQL baseline follows this structure directly; our contribution is layering a cascade on top of the semantic operator itself, once filtering has already made the pool as small as it can.

Avoiding one LLM call per row in the first place. Trummer [12] treats semantic filtering as a block-nested-loops join: the LLM gets a batch of rows from each side at once and returns every match in a single call, with an adaptive batch size chosen to fit the model’s context budget. Our Trummer baseline implements this algorithm directly; CASTRUMMER keeps its batching but adds structured filtering and calibrated escalation on top.

Combining evidence from more than one source. ELEET [13] treats a semantic scan as a multi-modal operator, reading structured and unstructured evidence for the same entity from separate sources before selecting. It also gives a cost-aware reordering rule for a sequence of operators, which we reuse (§4) to order the stages of CASTRUMMER.

Deciding when a cheap stage can be trusted. A hand-picked threshold gives no guarantee. Stretto [10] calibrates a cheap stage with a Beta posterior over a log-odds score and adds a KV-cache-aware execution ladder; we build on the former, extended to a score-stratified sample (§4.2), and skip the latter, which needs attention-state access our Ollama backend does not give. Other approaches calibrate on large validation sets with accuracy guarantees [?], isotonic regression [?], learned proxy scores [?] or

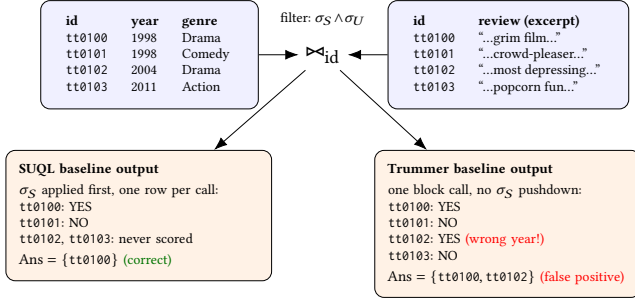


Figure 1: The running example as a join.

learned routers [?]; our setting must recalibrate for every question from a handful of labels.

Classical query optimization. Filtering ahead of an expensive operator goes back to cost-based join ordering [11], expensive-predicate scheduling [2, 4, 5] and the per-pair join cost it avoids [8]. Text-to-SQL systems [3] handle only the structured half, a gap TAG [1] names explicitly; LOTUS [9] and Palimpzest [6] come closest to us, optimizing a full plan of declarative semantic operators with cost-based model choice and batching, whereas we optimize structured and unstructured predicates in tandem.

3 PRELIMINARIES

Query model. Let $S(id, A_1, \dots, A_k)$ be a relation with structured attributes and $U(id, txt)$ a relation of unstructured text. A question $Q = (\sigma_S, \sigma_U)$ has a structured predicate σ_S , answerable in a DBMS, and a semantic predicate σ_U over txt , delegated to an LLM. Its answer is $\text{Ans}(Q) := \pi_{id}(\sigma_S(S) \bowtie_{id} \sigma_U(U))$. We assume the predicates are evaluated on disjoint evidence, so their selectivities are independent, and that σ_S is far cheaper than σ_U : all methods try to spend as few LLM calls on σ_U as possible. Predicate extraction, which maps a natural-language question to (σ_S, σ_U) , is a single LLM call shared by the four methods, which therefore differ only in how they compute $\text{Ans}(Q)$.

SUQL. σ_S becomes a WHERE clause, yielding candidates $R_\sigma = \sigma_S(S)$; the LLM is called once per candidate through $\text{answer}(txt, \text{question})$, for $O(|R_\sigma|)$ calls. Each row is judged alone with its full text, so quality is close to the best achievable, but the cost cannot be reduced because no work is shared across rows.

TRUMMER. No structured filter is applied. The two relations are partitioned into blocks and each call receives one block of each, like a block nested-loop join; the model must match rows by id, evaluate σ_S without a query engine, and evaluate σ_U . This divides the number of calls by roughly the block size, but the three tasks in one call cost quality: the model tends to rely on the structured condition alone or on superficial keywords.

Example 3.1 (Running example). For the question above, Figure 1 shows four movies joined with one review each, one per row category used in our benchmark (§5.1): tt0100 (1998, “grim film”) is a *true positive*; tt0101 (1998, “crowd-pleaser”) satisfies σ_S but not σ_U ; tt0102 (2004, “most depressing”) satisfies only σ_U ; tt0103 (2011, “popcorn fun”) satisfies neither. SUQL applies σ_S first and judges only tt0100 and tt0101, so $\text{Ans} = \{\text{tt0100}\}$. TRUMMER

Algorithm 1 Cascade (CASUQL: $B=B'=1$; CASTRUMMER: batches B, B')

Require: rows R_σ passing σ_S ; batch sizes B, B' ; calibration size $|C|$; target Q^* , level α

- 1: score every row with Cheap in batches of $B \rightarrow \ell_r$
- 2: $C \leftarrow |C|$ rows stratified by ℓ ; $y_C \leftarrow \text{Expensive}(C)$ $\triangleright$ labels are kept as final answers
- 3: $(\tau_-, \tau_+) \leftarrow \text{FitBand}(\ell, y_C, Q^*, \alpha)$ $\triangleright$ §4.2
- 4: $\text{Ans} \leftarrow \{r \in C : y_r = \text{YES}\} \cup \{r \notin C : \ell_r \geq \tau_+\}$
- 5: **for** batch $b \subset \{r \notin C : \tau_- < \ell_r < \tau_+, |b| \leq B'\}$ **do**
- 6: $\text{Ans} \leftarrow \text{Ans} \cup \{r \in b : \text{Expensive}(b)_r = \text{YES}\}$ $\triangleright$ split and retry if the answer is unusable
- 7: **end for**
- 8: **return** Ans

hands all four rows to one call, which must match rows, apply σ_S and apply σ_U at once; a model that tracks the sentiment but drops the year check also accepts tt0102, a false positive.

4 CASCADED ARCHITECTURES

Both cascades share the structure of Algorithm 1. The structured filter runs first in the DBMS. A *cheap stage* then scores every surviving row with a log-odds $\ell = \log \frac{p}{1-p}$, a calibrated *band* (τ_-, τ_+) routes rows (accept if $\ell \geq \tau_+$, reject if $\ell \leq \tau_-$), and an *expensive stage* judges only the rows inside the band. A side of the band that calibration cannot certify is left open ($\tau_+ = +\infty$ or $\tau_- = -\infty$), so no row is decided by a cheap stage that has not been shown reliable.

CASUQL scores each row in one single-token call. The log-odds come from the answer-token log-probabilities: we pool all Yes-like and all No-like spellings among the ten most likely first tokens. The calibration rows keep their expensive answer, so no row is judged twice, and CASUQL never issues more expensive calls than SUQL.

CASTRUMMER changes three things in TRUMMER. (1) *Structured filter*: σ_S runs in the DBMS, so no context is spent on rows it rejects, and the false positive tt0102 of the example can no longer be admitted. (2) *Batched cheap pass*: one call per batch of B rows returns YES, NO or UNCERTAIN per row (mapped to $\ell = +2, 0, -2$); allowing UNCERTAIN sends rows with indirect evidence to the expensive stage instead of labeling them NO. In the example, tt0101 is rejected and the sarcastic review of tt0100 is escalated and accepted. (3) *Batched expensive verification*: escalated rows are verified in batches of B' . If a batch’s answer is unusable (the prompt overflowed the context and the model omitted row ids), it is split in two and each half retried, so no candidate is silently dropped; every retry counts as an expensive call.

Filtering with σ_S first is the classical rule of ordering selections by ascending $c_i/(1 - s_i)$ [5, 13]: σ_S is nearly free and selective.

4.1 Cost Analysis

We count LLM calls, which dominate the cost. Let $N = |S|$ and $|R_\sigma| = sN$. TRUMMER is a block nested-loop join [??] and issues $O(N^2/(b_S b_U))$ calls whatever s is. The other methods are linear in sN : SUQL issues sN calls, CASUQL sN cheap and $O(e sN)$ expensive calls, and CASTRUMMER $O(sN/B)$ cheap and $O(e sN/B')$ expensive calls, where e is the escalated fraction. Calibration calls

are not extra, since the calibration rows keep their expensive answer. The cascade saves expensive calls on the fraction $d = 1 - e$ of rows the cheap stage decides alone. In wall-clock time, a row-wise cascade pays t_c per row for the cheap call and t_e for the escalated rows, against t_e per row for SUQL, so it is faster exactly when

$$t_c/t_e < d.$$

4.2 Calibrating the Cascade

For each question, $|C|$ candidates are labeled by the expensive LLM. The sample is stratified by cheap score: the candidates are cut into $K=4$ equal-count strata (equal scores never straddle a cut) and the labels are spread evenly over them.

Beta-posterior bounds. Let stratum h have N_h candidates, n_h of them labeled, y_h of those positive. With a uniform prior, Beta-Binomial conjugacy [?] gives the positive rate $\pi_h \sim \text{Beta}(1+y_h, 1+n_h-y_h)$. Labeled rows keep their answer, so only the $U_h = N_h - n_h$ unlabeled rows can be lost or wrongly accepted; their positives number $X_h \sim \text{Bin}(U_h, \pi_h)$. Rejecting the b lowest strata leaves a recall, relative to the expensive model, of

$$\rho_b = 1 - \frac{\sum_{h \leq b} X_h}{Y + \sum_h X_h},$$

with $Y = \sum_h y_h$, and accepting strata $h \geq t$ without verification has precision $\omega_t = \sum_{h \geq t} X_h / \sum_{h \geq t} U_h$. We propagate the posteriors to both by Monte Carlo (2,000 draws) and take lower credible bounds at level $\alpha=0.9$: $\hat{r}_\alpha(b) = Q_{1-\alpha}(\rho_b)$ and $\hat{p}_\alpha(t) = Q_{1-\alpha}(\omega_t)$, where Q_q is the q -quantile. With one stratum and a uniform sample they reduce to the Beta($1+TP$, $1+FN$) and Beta($1+TP$, $1+FP$) bounds. The band rejects the largest prefix of strata and accepts the largest suffix that clear the target Q^* :

$$\begin{aligned} \tau_- &= \text{top score of stratum } \max\{b : \hat{r}_\alpha(b) \geq Q^*\}, \\ \tau_+ &= \text{lowest score of stratum } \min\{t : \hat{p}_\alpha(t) \geq Q^*\}. \end{aligned}$$

Anything not certified is escalated.

Target and power. We use $Q^*=0.8$; 0.9 cannot be certified from 20 labels, since a cheap stage correct on all 20 has a lower bound of only 0.896. In simulations (300 pools per configuration, 40 to 1,000 rows, 10–30% positives, cheap-score AUC 0.76–0.96, $|C| \in \{20, 40, 100\}$; script in the artifact) the true recall reached Q^* in at least 99% of the trials with a certified reject region, against a target of 90%. The price is modest power: with $|C| = 20$ the cheap stage decides on its own 8–13% of the rows (1% with 10% positives), 28–36% with $|C| = 40$, and 56% of a 1,000-row pool needs $|C| = 100$.

5 EXPERIMENTS

5.1 Setup

Data and questions. *IMDb*: 25k movies (title, director, year, runtime, genres) and one review each. *Amazon Fashion*: 186k products and 882k reviews (2018 McAuley Lab release), a cross-domain check. Each dataset has ten questions (10q), each evaluated on its own pool of 100 items. On *IMDb* a pool holds the four row kinds of Example 3.1: 12 answers, 28 rows satisfying only σ_S , 30 only σ_U and 30 neither, so 40 rows pass σ_S . On *Amazon Fashion* 50 products pass σ_S , 15 of them answers; the ten questions are a held-out set, disjoint from the smaller suites used during development. *IMDb* questions combine one or two structured conditions (genre, year,

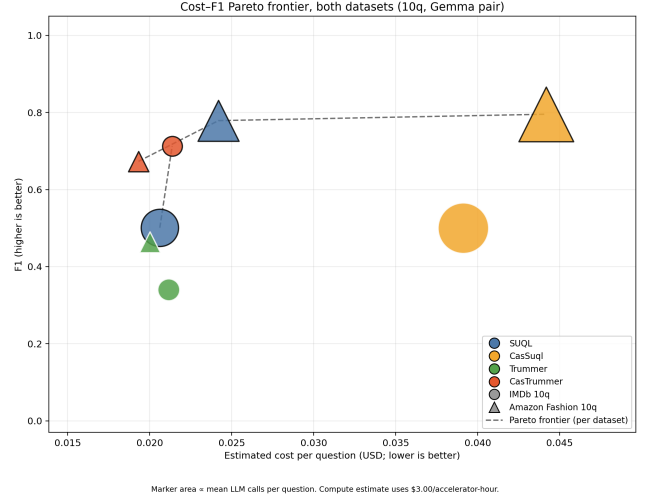


Figure 2: Cost–F1 trade-off on *IMDb* (circles) and *Amazon Fashion* (triangles).

runtime) with one semantic condition on the reviews; the ten questions span seven genres and range from direct cues (*funny*) to abstract, paraphrase-prone ones (*chemistry*, *approval of a twist*). *Amazon Fashion* questions restrict the product type through its title and ask about a property described in the reviews. Ground truth is the conjunction of σ_S and hand-authored regular-expression patterns encoding the semantic condition; no LLM judges. Labels are therefore deterministic, but a correct paraphrase outside the patterns counts as a negative, so absolute F1 values are best read as comparisons between methods.

Metrics, models, parameters. Quality is precision, recall and F1 of the returned ids; cost is LLM calls (cheap and expensive), wall-clock time and dollars, from accelerator time at \$3/hour. We report the mean and standard deviation over the ten questions of the per-question mean of ten repetitions; as all calls are at temperature 0, repetitions differ only in the calibration sample. All methods use Gemma 4 e2b (cheap) and Gemma 4 26b (expensive), served by Ollama on one NVIDIA H200; the baselines use only the expensive model. SUQL and CASUQL process one row per call; CASTRUMMER uses $B=8$ and $B'=32$, and TRUMMER the 4,096-token budget of its optimizer [12]. Both cascades use $|C|=20$, $K=4$, $\alpha=0.9$, $Q^*=0.8$. Cheap scores of CASUQL are read from Ollama’s native log-probabilities (its OpenAI-compatible endpoint returns none). Reviews are truncated before entering a prompt: to 1,800 characters for the cheap stage of CASUQL, 3,500 for both stages of CASTRUMMER, 6,000 for the expensive stage of CASUQL and SUQL, and 1,400 for TRUMMER; the same limits apply to *Amazon Fashion*, whose reviews are much shorter (median 270 characters, against 1,115 on *IMDb*). Code, suites, prompts and scripts: <https://github.com/AnnaRemi/SemanticCascadedLLMJoins>.

5.2 Results

Quality. CASTRUMMER raises the F1 of TRUMMER from 0.340 to 0.713 on *IMDb* (+110%, recall 0.275 to 0.708) and from 0.464 to 0.673 on *Amazon Fashion* (+45%, mostly through precision, 0.565

Metric	IMDb 10q					Amazon Fashion 10q				
	SUQL	CasSUQL	TRUMMER	CasTRUMMER	CasT ⁻	SUQL	CasSUQL	TRUMMER	CasTRUMMER	CasT ⁻
Precision	0.818 (0.213)	0.830 (0.196)	0.620 (0.313)	0.779 (0.186)	0.788 (0.230)	0.820 (0.115)	0.839 (0.113)	0.565 (0.200)	0.755 (0.285)	0.673 (0.252)
Recall	0.400 (0.207)	0.400 (0.203)	0.275 (0.193)	0.708 (0.189)	0.725 (0.208)	0.807 (0.254)	0.807 (0.234)	0.473 (0.260)	0.647 (0.325)	0.633 (0.360)
F1	0.501 (0.168)	0.500 (0.153)	0.340 (0.182)	0.713 (0.131)	0.723 (0.165)	0.779 (0.142)	0.796 (0.133)	0.464 (0.187)	0.673 (0.294)	0.621 (0.303)
Wall time (s)	24.7 (0.9)	47.0 (2.4)	25.4 (2.6)	25.7 (2.6)	22.3 (2.2)	29.0 (2.1)	53.1 (2.9)	24.0 (5.4)	23.2 (8.4)	21.4 (9.1)
LLM calls	40.0 (0.0)	78.0 (2.8)	10.5 (1.3)	7.6 (0.7)	9.0 (0.0)	49.4 (0.5)	92.0 (3.7)	9.0 (2.4)	8.2 (3.0)	9.0 (3.4)
Cost (\$)	0.0206 (0.0008)	0.0391 (0.0020)	0.0212 (0.0022)	0.0214 (0.0022)	0.0186 (0.0018)	0.0242 (0.0017)	0.0442 (0.0024)	0.0200 (0.0045)	0.0193 (0.0070)	0.0178 (0.0076)

Table 1: Quality and cost per question, mean (std. dev.) over the 10 questions, 10 repetitions each. Bold: best of the four methods. CasT⁻: CasTRUMMER without the cheap stage ($|C|=0$, so it decides no row; ablation).

F1	IMDb 10q				Amazon Fashion 10q			
	SUQL	CasSUQL	TRUMMER	CasTRUMMER	SUQL	CasSUQL	TRUMMER	CasTRUMMER
q1	0.59	0.50	0.15	0.82	0.57	0.64	0.21	0.32
q2	0.71	0.71	0.55	0.62	0.85	0.93	0.43	0.93
q3	0.29	0.29	0.13	0.83	0.61	0.61	0.43	0.67
q4	0.47	0.47	0.57	0.92	0.94	0.94	0.79	0.90
q5	0.80	0.80	0.22	0.74	0.61	0.67	0.32	0.67
q6	0.45	0.45	0.15	0.55	0.91	0.91	0.61	0.80
q7	0.59	0.50	0.38	0.56	0.73	0.77	0.51	0.78
q8	0.40	0.40	0.25	0.83	0.85	0.85	0.57	0.85
q9	0.38	0.38	0.59	0.63	0.79	0.71	0.19	0.00
q10	0.33	0.50	0.40	0.64	0.94	0.94	0.58	0.83

Table 2: F1 per question (mean of 10 repetitions); bold: best of the four methods on that question.

to 0.755). The paired gain over the ten questions is $+0.373 \pm 0.076$ (better on all ten) and $+0.209 \pm 0.057$ (better on nine), and CasTRUMMER is the best (or tied best) of the four methods on seven (*IMDb*) and five (*Amazon Fashion*) questions (Table 2). CasSUQL reproduces the quality of SUQL (F1 0.500 against 0.501, paired difference -0.001 ± 0.022 ; 0.796 against 0.779, $+0.017 \pm 0.015$) with exactly the same recall (0.400; 0.807), and is never more than 0.05 F1 below SUQL on eight (*IMDb*) and nine (*Amazon Fashion*) questions.

Calls and cost. CasTRUMMER issues 81.1% (*IMDb*) and 83.3% (*Amazon Fashion*) fewer LLM calls than SUQL and 28.0% and 8.4% fewer than TRUMMER. Its wall-clock time equals that of SUQL on *IMDb* (+3.7%) and is 20.1% lower on *Amazon Fashion*, where it is also the cheapest method in dollars (-20% against SUQL). CasSUQL sends 5.0% and 13.8% fewer rows to the expensive model than SUQL. In Figure 2, CasTRUMMER and SUQL form the Pareto front on both datasets: CasTRUMMER reaches F1 0.713 against 0.501 on *IMDb* at a cost 4% higher, and is the cheapest on *Amazon Fashion*, where SUQL keeps the higher F1 (0.779 against 0.673).

The calibration does what it certifies. CasSUQL certified a reject region in 34% (*IMDb*) and 77% (*Amazon Fashion*) of its repetitions, and recall did not drop in any dataset; when the sample cannot certify a band the cascade falls back to the expensive model instead of guessing. CasSUQL collapsed on no question (no F1 of 0 where SUQL exceeded 0.3), and no row was judged twice.

Where the gain comes from, and what the cheap stage costs. The cheap stage decided on its own only 2.0 of 40 rows (*IMDb*) and 7.4 of 50 (*Amazon Fashion*) for CasSUQL, and 1.4 and 0.9 for CasTRUMMER. Column CasT⁻ of Table 1 runs CasTRUMMER with $|C|=0$, so that every row passing σ_S is verified by the expensive model in batches. On *IMDb* it is as accurate (F1 0.723 against 0.713; paired difference -0.010 ± 0.035) and 3.4 s faster; on *Amazon Fashion* CasTRUMMER is better by 0.052 ± 0.026 at equal

Cheap $\rightarrow$ expensive	AUC	t_c (s)	t_e (s)	t_c/t_e
Gemma 4 e2b $\rightarrow$ 26b (used here)	0.85	0.42	0.65	0.65
Gemma 4 e2b $\rightarrow$ 31b	0.85	0.42	0.79	0.54
Qwen 3.5 2b $\rightarrow$ 27b	0.83	0.36	1.00	0.36
Llama 3.1 8b $\rightarrow$ 70b	0.81	0.23	0.78	0.30

Table 3: Stand-alone latency of candidate model pairs on *IMDb* rows (single stream, H200): t_c is a one-token scoring call of the cheap model, t_e an answer call of the expensive one; AUC is that of the cheap model’s log-odds against the ground truth (120 rows). Only the first pair is evaluated in this paper.

recall, which we cannot separate from an effect of batch partitioning. The quality gain over TRUMMER (+0.38 and +0.16 without the cheap stage) therefore comes from the other parts of the CasTRUMMER architecture, structural filtering and batched expensive verification with retry; the calibrated cheap stage adds a recall guarantee without a measurable quality cost, at 1.8–3.4 s of extra wall-clock time. For CasSUQL the cost model of \$4.1 explains the time: a cheap call took 0.58 s against 0.62 s for an expensive one on *IMDb* (0.55 against 0.59 s on *Amazon Fashion*), so $t_c/t_e \approx 0.94$ while $d \leq 0.15$, and CasSUQL takes 1.8 to 1.9 times the time of SUQL. Stand-alone measurements of other pairs (Table 3) show wider gaps, down to $t_c/t_e = 0.30$ for Llama 3.1 8B and 70B, with cheap scores of comparable quality (AUC 0.81–0.85); there a row-wise cascade would need to decide only about a third of the rows to save time, which our simulations suggest a calibration set of about 40 rows can reach (§4.2). We have not yet evaluated these pairs.

Which cascade to use. CasTRUMMER is the choice when calls and dollars matter: it improves the F1 of TRUMMER on nine or ten

of the ten questions and issues about 80% fewer calls than *SUQL*, although on *Amazon Fashion* *SUQL* remains the more accurate. *CAS-SUQL* is the choice when the quality of row-wise judgments must be preserved with a certified bound on what the cheap stage rejects; it pays a time overhead until the cheap model is much faster than the expensive one.

Scope. We use one model pair, ten questions per dataset, pools of 40 and 50 candidates (so 20 calibration rows are 40–50% of a pool), and lexical ground truth. On one *Amazon Fashion* question the F1 of *CAS-TRUMMER* is 0, which drives its large deviation.

6 CONCLUSION

CAS-SUQL and *CAS-TRUMMER* run the structured filter in the DBMS and put a cheap stage with a statistically calibrated band in front of the semantic operators of *SUQL* and *TRUMMER*. The calibration is valid and conservative, wastes no label, and lets the cascades keep the recall of the expensive model; *CAS-TRUMMER* improves the F1 of *TRUMMER* by 45–110% and issues 81–83% fewer calls than *SUQL*. The cheap stage saves time only when it is much faster than the expensive one and certified on many rows ($t_c/t_e < d$). Future work targets exactly that regime: model pairs with a larger latency gap, calibration sets that grow with the pool (about 10% to decide half of it), larger pools, and more than two stages.

REFERENCES

- [1] Asim Biswal, Liana Patel, Siddharth Jha, Amog Kamsetty, Shu Liu, Joseph E. Gonzalez, Carlos Guestrin, and Matei Zaharia. 2024. Text2SQL is Not Enough: Unifying AI and Databases with TAG. *arXiv preprint arXiv:2408.14717* (2024).
- [2] Surajit Chaudhuri and Kyuseok Shim. 1996. Optimization of Queries with User-Defined Predicates. In *Proceedings of the 22nd International Conference on Very Large Data Bases (VLDB)*. 87–98.
- [3] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2023. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *arXiv preprint arXiv:2308.15363* (2023).
- [4] Joseph M. Hellerstein. 1998. Optimization Techniques for Queries with Expensive Methods. *ACM Transactions on Database Systems* 23, 2 (1998), 113–157.
- [5] Joseph M. Hellerstein and Michael Stonebraker. 1993. Predicate Migration: Optimizing Queries with Expensive Predicates. In *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*. 267–276.
- [6] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baille Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, and Gerardo Vitagliano. 2024. A Declarative System for Optimizing AI Workloads. *arXiv preprint arXiv:2405.14696* (2024). System later presented as Palimpsest at CIDR 2025.
- [7] Shicheng Liu, Jialiang Xu, Wesley Tjangnaka, Sina J. Semnani, Chen Jie Yu, and Monica S. Lam. 2023. *SUQL*: Conversational Search over Structured and Unstructured Data with Large Language Models. *arXiv preprint arXiv:2311.09818* (2023). Also in Findings of the Association for Computational Linguistics: NAACL 2024.
- [8] Priti Mishra and Margaret H. Eich. 1992. Join Processing in Relational Databases. *Comput. Surveys* 24, 1 (1992), 63–113.
- [9] Liana Patel, Siddharth Jha, Carlos Guestrin, and Matei Zaharia. 2024. LOTUS: Enabling Semantic Queries with LLMs Over Tables of Unstructured and Structured Data. *arXiv preprint arXiv:2407.11418* (2024).
- [10] Gabriele Sanmartino, Matthias Urban, Paolo Papotti, and Carsten Binnig. 2026. The Stretto Execution Engine for LLM-Augmented Data Systems. *arXiv preprint arXiv:2602.04430* (2026).
- [11] P. Griffiths Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. 23–34.
- [12] Immanuel Trummer. 2025. Implementing Semantic Join Operators Efficiently. *arXiv preprint arXiv:2510.08489* (2025).
- [13] Matthias Urban and Carsten Binnig. 2024. ELET: Efficient Learned Query Execution over Text and Tables. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4867–4880.